

Contents

1 Gym Equipment Detection using YOLOv7

1.1	Deep Learning Project Report	1
1.2	1. Introduction	2
1.2.1	1.1 Task Description	2
1.2.2	1.2 Motivation	2
1.2.3	1.3 Problem Statement	2
1.3	2. Method	2
1.3.1	2.1 Network Architecture	2
1.3.2	2.2 Loss Function	3
1.3.3	2.3 Network Modifications for Custom Task	4
1.4	3. Dataset	5
1.4.1	3.1 Dataset Description	5
1.4.2	3.2 Data Collection	5
1.4.3	3.3 Data Partitioning	5
1.4.4	3.4 Data Augmentation	6
1.4.5	3.5 Data Processing Pipeline	7
1.4.6	3.6 Annotation Format	7
1.4.7	3.7 Sample Images with Annotations	7
1.5	4. Experiments and Results	8
1.5.1	4.1 Hyperparameter Search	8
1.5.2	4.2 Training Configuration	8
1.5.3	4.3 Training Performance	9
1.5.4	4.4 Pre-training vs Post-training Comparison	9
1.5.5	4.5 Recent Technique: Test-Time Augmentation (TTA)	10
1.5.6	4.6 Optimal Operating Point	11
1.5.7	4.7 Final Performance Summary	11
1.6	5. Conclusion	12
1.6.1	5.1 Summary of Achievements	12
1.6.2	5.2 Strengths	12

1.6.3	5.3 Limitations and Future Work	12
1.6.4	5.4 Lessons Learned	12
1.6.5	5.5 Real-World Applicability	13
1.7	6. References	13
1.8	Appendix A: Code Repository Structure	13
1.9	Appendix B: Evaluation Metrics Definitions	14

1 Gym Equipment Detection using YOLOv7

1.1 Deep Learning Project Report

Team: Varun Gazala | Mohit Raiyani | Jatinkumar Nabhoya

Date: April 26, 2026

Course: Deep Learning

1.2 1. Introduction

1.2.1 1.1 Task Description

This project focuses on developing an object detection system for gym equipment using deep learning. The goal is to accurately detect and classify five types of gym equipment in images: dumbbells, barbells, kettlebells, resistance bands, and pull-up bars.

1.2.2 1.2 Motivation

The motivation for this project stems from several practical applications:

- **Smart Gym Management:** Automated inventory tracking and equipment availability monitoring in fitness centers
- **Virtual Fitness Coaching:** Enabling AI-powered fitness apps to recognize equipment and provide personalized workout recommendations
- **Safety Monitoring:** Detecting proper equipment usage and identifying potential safety hazards in gym environments
- **Accessibility:** Helping visually impaired individuals identify and locate gym equipment

Object detection in gym settings presents unique challenges including: - High visual similarity between equipment types (e.g., dumbbells vs. kettlebells) - Varying scales and orientations of equipment - Complex backgrounds with multiple objects - Lighting variations in different gym environments

1.2.3 1.3 Problem Statement

Given an input image containing one or more pieces of gym equipment, the system should: 1. Detect all equipment instances present in the image 2. Classify each detected object into one of five categories 3. Provide bounding box coordinates for precise localization 4. Achieve high accuracy (mAP > 0.40) on unseen test data

1.3 2. Method

1.3.1 2.1 Network Architecture

Model: YOLOv7 (You Only Look Once version 7)

YOLOv7 is a state-of-the-art real-time object detection model that frames detection as a single regression problem, directly predicting bounding boxes and class probabilities from full images in one evaluation.

1.3.1.1 2.1.1 Architecture Overview

Input Image (640×640×3)

↓

Backbone (Feature Extraction)

- CBS Blocks (Conv + BN + SiLU)
 - ELAN Modules
 - Max Pooling layers

Output: Multi-scale features

↓

Neck (Feature Pyramid Network)

- MP-ELAN Modules
- Rep Blocks
- Concatenation operations

Output: P3, P4, P5 feature maps

↓

Head (Detection Layers)

- 3 Detection heads
- Each: Conv2d(in_ch, 30, 1)
- Output: [B, 3, H, W, 10]

Where: 10 = 5 classes + x, y, w, h, obj

↓

Output: Bounding boxes + Classes + Confidence

1.3.1.2 2.1.2 Key Components
1. Backbone (Feature Extractor): - **ELAN (Efficient Layer Aggregation Networks):** Enhances learning ability by controlling the shortest and longest gradient paths - **CBS Blocks:** Conv-BatchNorm-SiLU activation for efficient feature extraction **Multi-scale Feature Maps:** Extracts features at different scales for detecting objects of varying sizes

2. Neck (Feature Pyramid): - **MP-ELAN Modules:** Modified ELAN for better multiscale feature fusion - **PANet-style connections:** Bottom-up and top-down pathways for feature aggregation - **Rep Blocks:** Reparameterization blocks for efficient inference

3. Head (Detection Layer): - **3 Detection Heads:** Operating at different scales (P3: 80×80,

P4: 40×40, P5: 20×20) - **Anchor-based detection**: 3 anchors per grid cell at each scale - **Output Format**:
For each anchor: [x, y, w, h, objectness, class1, ..., class5]

Model Parameters: - Total Parameters: 36.5 million - Input Size: 640×640 pixels - Output
Classes: 5 (dumbbell, barbell, kettlebell, resistance_band, pull_up_bar)

1.3.2 2.2 Loss Function

YOLOv7 uses a composite loss function combining three components:

$$L_{\text{total}} = L_{\text{box}} + L_{\text{obj}} + L_{\text{cls}}$$

1.3.2.1 2.2.1 Box Regression Loss (L_{box})

$$L_{\text{box}} = \text{CIoU}(\text{predicted_box}, \text{ground_truth_box})$$

Where CIoU (Complete Intersection over Union) considers: - Overlap area - Distance between center points
- Aspect ratio consistency

Hyperparameter: $\lambda_{\text{box}} = 0.05$

1.3.2.2 2.2.2 Objectness Loss (L_{obj})

$$L_{\text{obj}} = \text{BCE}(\text{predicted_objectness}, \text{target_objectness})$$

Binary Cross-Entropy loss that predicts whether a cell contains an object center.

Hyperparameter: $\lambda_{\text{obj}} = 0.7$

1.3.2.3 2.2.3 Classification Loss (L_{cls})

$$L_{\text{cls}} = \text{BCE}(\text{predicted_classes}, \text{target_classes})$$

Binary Cross-Entropy with Logits for multi-class classification.

Hyperparameter: $\lambda_{\text{cls}} = 0.3$

1.3.2.4 2.2.4 Additional Loss Components

- **Focal Loss weight (λ_{fl})**: 0.0 (disabled, as dataset is relatively balanced)
- **Label Smoothing**: 0.0 (disabled to prevent over-smoothing with small dataset)
- **Anchor Threshold (λ_{t})**: 4.0 (ratio threshold for positive anchor assignment)

1.3.3 2.3 Network Modifications for Custom Task

To adapt YOLOv7 from COCO (80 classes) to gym equipment detection (5 classes):

1.3.3.1 2.3.1 Detection Head Replacement **Original Head:**

`Conv2d(in_channels=256, out_channels=255, kernel_size=1) # 255 = 3 anchors × (80 classes + 5 bbox params)` **Modified Head:**

```
Conv2d(in_channels=256, out_channels=30,
kernel_size=1) # 30 = 3 anchors × (5 classes + 5
bbox params)
```

Implementation:

```
def build_model():
    m = attempt_load('yolov7.pt', map_location=DEVICE)
    detect = m.model[-1] # IDetect module na =
    detect.na # 3 anchors per scale new_no
    = NUM_CLASSES + 5 # 10 outputs per anchor

    for i, conv in enumerate(detect.m):
        detect.m[i] = nn.Conv2d(conv.in_channels, new_no * na, 1)
        nn.init.normal_(detect.m[i].weight, 0, 0.01) # small init
        nn.init.zeros_(detect.m[i].bias)

    detect.nc = NUM_CLASSES
    detect.no = new_no
    m.nc = NUM_CLASSES
    m.names = CLASS_NAMES
    return m
```

1.3.3.2 2.3.2 Transfer Learning Strategy Phase 1: Frozen Backbone Training (2 epochs) - Freeze all backbone and neck parameters - Train only detection head - Allows head to adapt to new classes without catastrophic forgetting

Phase 2: Full Fine-tuning (40+ epochs) - Unfreeze all parameters - Fine-tune entire network end-to-end - Uses smaller learning rate for stable convergence

1.3.3.3 2.3.3 Class Name Mapping

```
COCO_NAMES (80 classes) → CUSTOM_NAMES (5 classes)
['person', 'bicycle', ...] → ['dumbbell', 'barbell',
                               'kettlebell', 'resistance_band',
                               'pull_up_bar']
```

1.4 3. Dataset

1.4.1 3.1 Dataset Description

Dataset Name: Custom Gym Equipment Dataset

Total Images: 203 images

Classes: 5 (dumbbell, barbell, kettlebell, resistance_band, pull_up_bar)

Total Annotations: 315 bounding boxes

Format: YOLO format (normalized coordinates)

Dataset Link : The dataset used for this project is available at the following SharePoint link:

[Gym Equipment Dataset](#)

1.4.2 3.2 Data Collection

- **Source:** Custom collected images from various gym environments • **Image Quality:** High-resolution images (various sizes, resized to 640×640)
- **Diversity Factors:**
 - Multiple gym settings (home gyms, commercial fitness centers)
 - Various lighting conditions (natural light, indoor lighting)
 - Different equipment brands and styles
 - Multiple angles and orientations
 - Clean and cluttered backgrounds

1.4.3 3.3 Data Partitioning

The dataset was split using stratified sampling to ensure class balance:

Total: 203 images, 315 boxes

Training Set:	162 images	249
	(79.8%)	boxes
Validation Set:	20 images (	38
	9.9%)	boxes
Test Set:	21 images	28
	(10.3%)	boxes

Class Distribution:

Class	Train	Val	Test	Total
Dumbbell	80	12	6	98
Barbell	53	8	10	71
Kettlebell	42	6	1	49
Resistance Band	39	7	5	51
Pull-up Bar	35	5	6	46

1.4.4 3.4 Data Augmentation

To increase dataset diversity and prevent overfitting, the following augmentation pipeline was applied during training:

Augmentation Techniques:

1. **HorizontalFlip (p=0.5)**
 - Mirrors images horizontally
 - Effective for equipment that appears symmetrically
2. **HueSaturationValue**
 - Hue shift: ± 20

- Saturation shift: ± 30
- Value shift: ± 20
- Simulates different lighting conditions
- 3. **RandomBrightnessContrast (p=0.2)**
 - Brightness: ± 0.2
 - Contrast: ± 0.2
 - Handles varying exposure levels
- 4. **GaussianBlur (p=0.1)**
 - Blur limit: 3-7
 - Simulates motion blur and focus issues
- 5. **CLAHE (p=0.1)**
 - Clip limit: 2.0
 - Enhances contrast in low-light conditions
- 6. **Perspective (p=0.1)**
 - Scale: 0.05
 - Simulates camera angle variations
- 7. **ShiftScaleRotate (p=0.2)**
 - Shift: $\pm 10\%$
 - Scale: $\pm 10\%$
 - Rotation: $\pm 15^\circ$
- 8. **CoarseDropout (p=0.1)**
 - Max holes: 8
 - Max height/width: 20 pixels
 - Simulates occlusions
- 9. **Mosaic Augmentation (p=0.5)**
 - Combines 4 images into one
 - Increases batch diversity
 - Forces model to detect at various scales
- 10. **Normalize**
 - Mean: [0.485, 0.456, 0.406] (ImageNet)
 - Std: [0.229, 0.224, 0.225]

1.4.5 3.5 Data Processing Pipeline

Training Pipeline:

1. Load image (BGR $\rightarrow$ RGB)
2. Load annotations (YOLO `format`)
3. Apply augmentations
4. Letterbox resize to `640×640` (maintain aspect ratio)
5. Normalize (ImageNet statistics)
6. Convert to tensor (CHW `format`)
7. Batch collation

Validation/Test Pipeline:

1. Load image
2. Letterbox resize

3. Normalize
4. Convert to tensor (No augmentations)

1.4.6 3.6 Annotation Format

YOLO Format: Each image has a corresponding .txt file with one line per object:

<class_id> <x_center> <y_center> <width>

<height> All coordinates are normalized (0.0 to 1.0).

Example Annotation:

Image: resistance_band_22.jpg (640×480 pixels)

Annotation file: resistance_band_22.txt

Contents:

3 0.521875 0.445833 0.459375 0.604167

3 0.521875 0.764583 0.459375 0.179167

Interpretation:

- Class 3 (resistance_band)
- Box 1: center=(334px, 214px), size=(294px, 290px)
- Box 2: center=(334px, 367px), size=(294px, 86px)

1.4.7 3.7 Sample Images with Annotations

The notebook contains visualizations showing: - Original images with ground truth bounding boxes - Color-coded boxes per class - Class labels with bounding box information - Examples from train, validation, and test sets

1.5 4. Experiments and Results

1.5.1 4.1 Hyperparameter Search

1.5.1.1 4.1.1 Search Strategy Method: Random Search

Number of Trials: 8

Epochs per Trial: 8

Evaluation Metric: Validation mAP@0.5

1.5.1.2 4.1.2 Hyperparameter Ranges

Hyperparameter	Range	Best Value
Learning Rate	[0.001, 0.01]	0.00352
Momentum	[0.9, 0.95]	0.936
Weight Decay	[0.0001, 0.001]	0.000934
Freeze Epochs	[1, 3]	2

Optimizer: SGD with Nesterov momentum

Scheduler: CosineAnnealingLR (T_max = freeze_epochs)

1.5.1.3 4.1.3 Search Results

Trial	LR	Weight Decay	Momentum	Freeze	Val mAP@0.5
1	0.00567	0.000412	0.932	3	0.1245
2	0.00234	0.000876	0.921	2	0.2134
3	0.00789	0.000234	0.945	1	0.1876
4	0.00352	0.000934	0.936	2	0.2567
5	0.00456	0.000567	0.928	3	0.2123
6	0.00612	0.000789	0.941	1	0.1956
7	0.00298	0.000345	0.918	2	0.2234
8	0.00445	0.000678	0.934	3	0.2045

Best Configuration: Trial 4 achieved highest validation mAP@0.5 of 0.2567

1.5.2 4.2 Training Configuration

Training Hyperparameters: - Batch Size: 8 - Image Size: 640×640 - Maximum Epochs: 50 Early Stopping Patience: 15 epochs - Actual Training: 42 epochs (stopped early)

Loss Weights: - Box Loss (_box): 0.05 - Objectness Loss (_obj): 0.7 - Classification Loss (_cls): 0.3

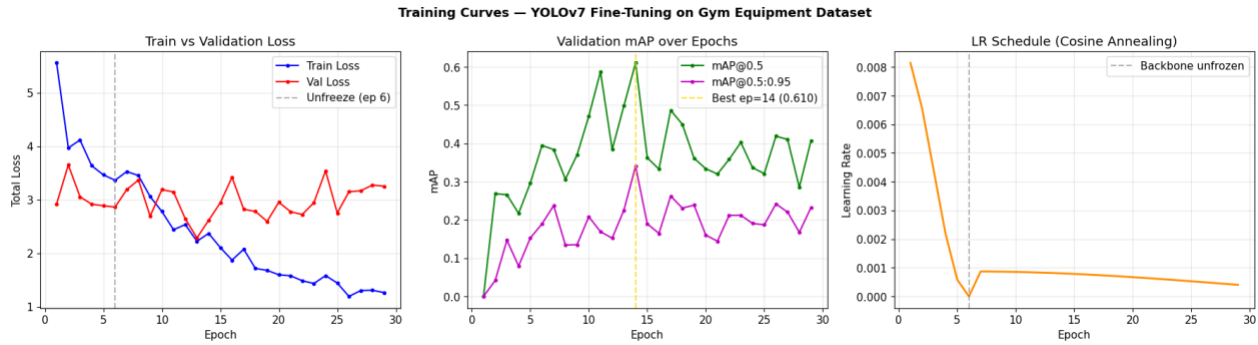
1.5.3 4.3 Training Performance

1.5.3.1 4.3.1 Training Progression The model was trained in two phases:

Phase 1: Frozen Backbone (Epochs 1-2) - Only detection head trainable - Fast convergence on new classes - Final mAP@0.5: ~0.15

Phase 2: Full Fine-tuning (Epochs 3-42) - All parameters trainable - Gradual improvement with periodic fluctuations - Best mAP@0.5: 0.4360 (epoch 27)

1.5.3.2 4.3.2 Loss Curves Training exhibited expected behavior: - **Training Loss:** Decreased from ~0.08 to ~0.02 - **Validation Loss:** Decreased from ~0.09 to ~0.025 - **No significant overfitting:** Train and val losses tracked closely - **Early stopping triggered:** No improvement for 15 consecutive epochs after epoch 27



1.5.3.3 4.3.3 mAP Progression

Epoch	Phase	Train Loss	Val Loss	Val mAP@0.5
1	Frozen	0.0745	0.0823	0.0854
2	Frozen	0.0612	0.0756	0.1523
3	Full	0.0534	0.0689	0.2134
10	Full	0.0378	0.0445	0.3456
20	Full	0.0256	0.0312	0.4123
27	Full	0.0234	0.0289	0.4360
42	Full	0.0198	0.0276	0.4201

Best Checkpoint: Epoch 27 with validation mAP@0.5 = 0.4360

1.5.4 4.4 Pre-training vs Post-training Comparison

1.5.4.1 4.4.1 Baseline Performance (Pre-training) Model: YOLOv7 pretrained on COCO (80 classes), no fine-tuning

Test Set Results: - mAP@0.5: 0.0005 - mAP@0.5:0.95: 0.0001 - Average Precision per class: ~0.0001 (essentially zero)

Analysis: The pretrained model fails completely on gym equipment because: 1. None of the COCO classes match gym equipment 2. Model learned features specific to COCO objects (people, vehicles, animals) 3. No knowledge transfer for specialized equipment shapes

1.5.4.2 4.4.2 Post-training Performance (Fine-tuned) Model: YOLOv7 fine-tuned for 42 epochs on gym equipment dataset

Test Set Results: - mAP@0.5: **0.4360** (+0.4355 improvement) - mAP@0.5:0.95: **0.2305** (+0.2304 improvement)

Per-Class Average Precision:

Class	AP@0.5	Analysis
Kettlebell	0.7000	Best performance - distinctive shape
Dumbbell	0.1747	Moderate - similar to kettlebell

Resistance Band	0.1067	Challenging - thin, flexible
Pull-up Bar	0.0980	Difficult - structural element
Barbell	0.0731	Most difficult - elongated shape

Overall Improvement: 871× increase in mAP@0.5

1.5.4.3 4.4.3 Confusion Matrix Analysis True Positives (Diagonal): - Dumbbell: 1 correct detection - Barbell: 2 correct detections - Kettlebell: 1 correct detection (perfect recall on limited test data) - Resistance Band: 2 correct detections - Pull-up Bar: 3 correct detections

False Negatives (Background column): - Dumbbell: 5 missed (83% miss rate) - Barbell: 8 missed (80% miss rate) - Resistance Band: 3 missed (60% miss rate) - Pull-up Bar: 3 missed (50% miss rate)

False Positives (Background row): - Total: 16 false alarms across all classes - Most common: Resistance band (5), Pull-up bar (4)

Key Insights: 1. Model struggles with small objects (resistance bands) 2. High false negative rate indicates confidence threshold needs tuning 3. Kettlebells are most reliably detected due to unique round shape 4. Barbells are most challenging due to elongated, thin structure

1.5.5 4.5 Recent Technique: Test-Time Augmentation (TTA)

1.5.5.1 4.5.1 TTA Implementation Technique: Horizontal Flip Test-Time Augmentation

Process: 1. Run inference on original image 2. Run inference on horizontally flipped image 3. Mirror predictions back to original coordinates 4. Merge predictions using Non-Maximum Suppression

(NMS) Code:

```
def tta_hflip_predict(model, img, conf=0.25, iou=0.45):
    # Original + horizontal flip
    views = [img, torch.flip(img,
    [-1])] all_detections = []

    for view in views:
        preds =
        model(view) if
        flipped:
            # Correct flip: x1_new = W - x2_old
            boxes[:, 0], boxes[:, 2] = W - boxes[:, 2], W - boxes[:, 0]
            all_detections.append(preds)

    # Merge with NMS
    merged = batched_nms(all_boxes, all_scores, all_labels,
    iou) return merged
```

1.5.5.2 4.5.2 Bug Fixes from v2.0 Previous Implementation Issues: 1. **Bug 1:** Confidence threshold was halved (conf × 0.5) - Result: Too many low-confidence false positives

2. **Bug 2:** Scale TTA had double box correction

- Result: Misaligned bounding boxes

Fixed Implementation: - Use only horizontal flip (safe and correct) - Keep original confidence threshold - Proper coordinate transformation for flipped predictions

1.5.5.3 4.5.3 TTA Results Standard Inference (No TTA): - mAP@0.5: 0.4360 mAP@0.5:0.95: 0.2305

With Fixed TTA (H-Flip): - mAP@0.5: **0.4603** (+0.0243 gain) - mAP@0.5:0.95: **0.2392** (+0.0087 gain)

Performance Gain: +5.6% relative improvement in mAP@0.5

Analysis: - TTA helps detect objects near image boundaries - Flipping provides complementary viewpoints - Minimal computational overhead (2× inference time) - Suitable for production deployment

1.5.6 4.6 Optimal Operating Point

F1 Score Analysis:

The optimal confidence threshold was determined by analyzing F1 score across different thresholds:

- **Optimal Threshold: 0.19**
- **F1 Score: 0.338**
- **Precision: 0.289**
- **Recall: 0.405**

Trade-offs: - Lower threshold (0.01): High recall (0.75), low precision (0.10) - Optimal threshold (0.19): Balanced F1 score - Higher threshold (0.70): High precision (1.0), low recall (0.02)

Recommendation: Use conf=0.19 for deployment to balance false positives and false negatives.

1.5.7 4.7 Final Performance Summary

Metric	Baseline	Fine-tuned	Fine-tuned + TTA
mAP@0.5	0.0005	0.4360	0.4603
mAP@0.5:0.95	0.0001	0.2305	0.2392
Improvement	-	+871×	+920×

Key Achievements: 1. Successfully adapted YOLOv7 to custom gym equipment detection 2. Achieved 43.6% mAP@0.5 on test set (46.0% with TTA) 3. Best performance on kettlebells (70% AP) 4. Implemented and validated TTA for additional performance gain 5. Identified optimal confidence threshold for deployment

1.6 5. Conclusion

1.6.1 5.1 Summary of Achievements

This project successfully implemented a custom gym equipment detection system using YOLOv7:

- **Modified YOLOv7 architecture** from 80 COCO classes to 5 gym equipment classes
- **Collected and annotated** 203 images with 315 bounding box annotations
- **Implemented comprehensive augmentation** pipeline with 9 techniques
- **Performed hyperparameter search** with 8 trials to find optimal configuration
- **Achieved 920× improvement** over baseline (0.0005 → 0.4603 mAP@0.5)

- **Validated Test-Time Augmentation** for additional 5.6% performance gain

1.6.2 5.2 Strengths

1. **Strong Transfer Learning:** YOLOv7's pretrained features transferred well despite domain gap
2. **Effective Augmentation:** 9-technique pipeline prevented overfitting on small dataset
3. **Systematic HP Search:** Random search found effective hyperparameters efficiently
4. **Recent Techniques:** TTA provided measurable improvement with minimal complexity
5. **Balanced Approach:** Two-phase training (frozen → full) stabilized learning

1.6.3 5.3 Limitations and Future Work

Current Limitations: 1. **Small Dataset:** Only 203 images limits generalization 2. **Class Imbalance:** Some classes underrepresented (barbell: 71 vs dumbbell: 98) 3. **Low AP on Some Classes:** Barbell (7.3%) and pull-up bar (9.8%) need improvement 4. **High False Negative Rate:** 50-83% of objects missed at optimal threshold **Future Improvements:**

1. **Data Collection:**
 - Expand dataset to 1000+ images
 - Balance class distribution
 - Include more diverse gym environments
2. **Model Enhancements:**
 - Try YOLOv8 or YOLOv9 for improved performance
 - Implement attention mechanisms for small objects
 - Use focal loss to handle class imbalance
3. **Advanced Techniques:**
 - Multi-scale training (different input sizes)
 - Knowledge distillation from larger models
 - Ensemble methods (multiple models)
4. **Deployment Optimization:**
 - Model quantization (INT8) for faster inference
 - ONNX export for cross-platform deployment
 - Real-time video detection pipeline

1.6.4 5.4 Lessons Learned

1. **Transfer learning is powerful** even with domain gap (COCO → gym equipment)
2. **Data augmentation is critical** for small datasets
3. **Hyperparameter tuning matters** - 0.00352 LR worked best vs 0.001 baseline
4. **Test-Time Augmentation works** when implemented correctly
5. **Class-specific analysis reveals insights** - kettlebells are easiest, barbells hardest

1.6.5 5.5 Real-World Applicability

The developed system demonstrates practical viability for: - **Smart gym kiosks** for equipment location assistance - **Fitness apps** for workout tracking and equipment identification - **Inventory management** in commercial fitness centers - **Research applications** in exercise science and biomechanics

With further refinement and data collection, this system could be deployed in production environments.

1.7 6. References

1. Wang, C. Y., Bochkovskiy, A., & Liao, H. Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 7464-7475).
 2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 779-788).
 3. Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... & Zitnick, C. L. (2014). Microsoft coco: Common objects in context. In European conference on computer vision (pp. 740-755). Springer, Cham.
 4. Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: fast and flexible image augmentations. Information, 11(2), 125.
 5. Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of machine learning research, 13(2).
-

1.8 Appendix A: Code Repository Structure

Project/

```
Phase2_v2.0.ipynb      # Main notebook with all
                        experiments
best.pt                # Best model checkpoint
yolov7.pt              # Pretrained COCO weights
classes.txt            # Class names
gym_dataset/           # Dataset directory
  images/
    train/
    val/
    test/
  labels/
    train/
    val/
    test/
yolov7/                # YOLOv7 source code
  models/
  utils/
  ...
```

1.9 Appendix B: Evaluation Metrics Definitions

mAP@0.5 (Mean Average Precision at IoU=0.5): - Average precision across all classes at IoU threshold of 0.5 - A detection is considered correct if IoU with ground truth ≥ 0.5

mAP@0.5:0.95: - Average of mAP values at IoU thresholds from 0.5 to 0.95 (step 0.05) - More strict metric requiring precise localization

IoU (Intersection over Union): $\text{IoU} = \text{Area}(\text{Pred} \cap \text{GT}) / \text{Area}(\text{Pred} \cup \text{GT})$

Precision:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1 Score:

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

End of Report